

Mastering Reinforcement Learning Basics & Continuous MDPs

Complete Foundations: Markov Decision Processes, The Markov Chain View, The Universal Algorithm Anatomy, Continuous Gaussian Policies, Bellman Self-Consistency, and Tensor Broadcasting Traps

Instructor: Prof. Sergey Levine (UC Berkeley) **Curriculum:** Berkeley CS285 + Achiam (Spinning Up) + Sutton & Barto **Scope:** General Continuous RL Theory & Robotics Slicing

0. The "Mental Map": Why Does Lecture 4 Exist?

In previous lectures, we examined **Imitation Learning** (Behavioral Cloning). Imitation learning is straightforward: record a human operator and train a neural network to mimic human control actions via supervised learning. But imitation learning has severe limits:

- What if you do not have 500 hours of perfect human demonstrations?
- What if a human operator cannot react quickly enough to regulate high-frequency (1 kHz) contact impedance during fracture events?
- What if the human demonstrator is sub-optimal?

This is why Reinforcement Learning exists. Instead of copying a human, the agent discovers optimal behavior autonomously through trial, error, and physical reward signals. Lecture 4 provides the foundational mathematical engine of RL: *Markov Decision Processes, Cumulative Return Objectives, Algorithm Anatomy, Value Functions, and Bellman Self-Consistency.*

1. MDPs & States

Markov Chains to MDPs
State Sufficiency

2. The Objective

Expected Return $J(\theta)$
Markov Chain View

3. Algorithm Anatomy

Sample $\rightarrow$ Evaluate $\rightarrow$
Improve Iterative Loop

4. Bellman Consistency

The 4 Value Functions
1-Step Lookahead Target

5. RL Taxonomy

Sample Efficiency vs.
Stability Trade-Off

Part 1: The Markov Decision Process (MDP) De-Mystified

(Slides 3–9) To optimize decisions with mathematics, we must model the world as a mathematical object. Prof. Levine builds this up in three progressive stages:

1. **Markov Chain:** A sequence of random states $s_1, s_2, \dots$ governed by a transition matrix $\mathcal{P}(s_{t+1}|s_t)$. No actions, no rewards—just physics rolling forward.
2. **Markov Decision Process (MDP):** Add an agent that injects decisions $a_t \sim \pi(a_t|s_t)$ and receives scalar rewards $r(s_t, a_t)$.
3. **Partially Observable MDP (POMDP):** The agent cannot see the true state s_t ; it only receives imperfect sensory observations $o_t \sim \mathcal{O}(o_t|s_t)$.

1.1 The "Goldfish Memory" Rule: The Markov Property

THE FORMAL MARKOV PROPERTY

A system satisfies the Markov Property if the transition probability to the next state depends **only on the current state and action**, and is conditionally independent of all historical states and actions:

$$\Pr(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = \Pr(s_{t+1} \mid s_t, a_t)$$

ROBOTICS IMPLICATION: STATE REPRESENTATION ENGINEERING

If you only feed your neural network the robot blade's Cartesian position (x, y, z) , **you violate the Markov property!** Why? Because seeing a blade at height $z = 3$ cm tells the policy nothing about whether the knife is currently speeding downward at 50 mm/s or retracting upward. To make the state strictly Markovian, you must include **first-order velocities** $(\dot{x}, \dot{y}, \dot{z})$, joint angular rates $\dot{q}$, and contact force derivatives $\dot{F}_z$.

Part 2: Defining the RL Objective & The Markov Chain View

(Slides 10–16) What does it mean for an algorithm to "learn"? It means tuning policy network parameters θ to maximize the expected total return over entire trajectories τ :

$$\theta^* = \arg \max_{\theta} J(\theta) = \arg \max_{\theta} \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^T r(s_t, a_t) \right]$$

2.1 Why Immediate Greedy Actions Fail

In Slide 11, Levine illustrates this with an autonomous vehicle crash: if a car is traveling at 120 km/h toward a concrete wall, slamming the brakes 2 meters away is the "best" immediate action, but the vehicle will still crash! The fatal error was committed 10 seconds earlier when the car decided to accelerate into a blind corner.

ROBOTICS SLICING: THE CUTICLE RUPTURE ANALOGY

When slicing soft fruit or biological tissue, the blade presses against elastic skin. If the policy acts greedily to maximize instantaneous downward penetration, it pushes down with 15 Newtons. At step $t = 50$, the cuticle suddenly fractures. Because mechanical resistance vanishes in under 5 milliseconds, the knife violently accelerates downward, crushing the specimen. **RL prevents this:** It trains the policy to modulate impedance, initiate sawing motions, and increase damping *before* the fracture point because RL optimizes for the **cumulative trajectory reward**, not the instantaneous step reward.

2.2 The Markov Chain View: Trajectory Distribution vs. State Marginal

In Slide 12, Levine asks: "*Is there another way to write the objective without sampling entire trajectories?*" By the **linearity of expectation**, the expectation of a sum equals the sum of the expectations:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^T r(s_t, a_t) \right] = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim p_{\theta}(s_t, a_t)} [r(s_t, a_t)]$$

In infinite-horizon discounted problems, we can define the **discounted state-action stationary distribution**:

$$\rho_{\theta}(s, a) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s, a_t = a \mid \pi_{\theta})$$

This allows rewriting the entire RL objective as an expectation over a single stationary distribution:

$$J(\theta) = \frac{1}{1 - \gamma} \mathbb{E}_{(s, a) \sim \rho_{\theta}(s, a)} [r(s, a)]$$



2.3 Continuous Action Parametrization & The Tanh Change of Variables

In physical robotics, actions are continuous torques or velocity setpoints $a \in \mathbb{R}^{d_a}$. We parameterize the policy as a multivariate Gaussian $\pi_\theta(u|s) = \mathcal{N}(\mu_\theta(s), \Sigma_\theta(s))$. To enforce real-world physical limits (e.g. joint torques bounded in $[-1, 1]$), we apply an invertible nonlinear squashing function: $a = \tanh(u)$.

MATHEMATICAL DERIVATION: DENSITY UNDER TANH SQUASHING

By the multidimensional **Change of Variables Theorem** for continuous random variables:

$$P(a | s) = P(u | s) \cdot \left| \det \left(\frac{\partial a}{\partial u} \right) \right|^{-1}$$

Since the squashing transformation $a_i = \tanh(u_i)$ acts element-wise, the Jacobian matrix $J = \frac{\partial a}{\partial u}$ is purely diagonal:

$$J_{ii} = \frac{d}{du_i} \tanh(u_i) = 1 - \tanh^2(u_i)$$

The determinant of a diagonal matrix is simply the product of its diagonal entries:

$$\det(J) = \prod_{i=1}^{d_a} (1 - \tanh^2(u_i))$$

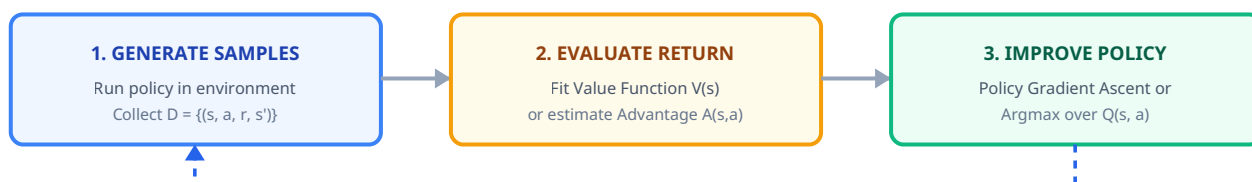
Taking the natural logarithm yields the exact log-probability correction:

$$\log \pi(a | s) = \log \mu(u | s) - \sum_{i=1}^{d_a} \log (1 - \tanh^2(u_i)) + \epsilon$$

where $\epsilon = 10^{-6}$ provides numerical stabilization against floating point underflow.

Part 3: The Anatomy of an RL Algorithm

(Slides 17–21) Every RL algorithm operates on a 3-step engine:



3.1 Where is the Computational Bottleneck?

- **On a Real Physical Robot:** Step 1 (Sampling) is the catastrophic bottleneck. Time runs at $1 \times$ real speed. Collecting 100,000 physical cuts requires **11.5 days** of non-stop operation, blade wear, and mechanical overheating.

- **In GPU Physics Simulators (Isaac Lab / Isaac Gym):** Step 1 is parallelized across **4,096 parallel GPU environments**, running at **10,000x real time**. The bottleneck shifts entirely to Step 2 and Step 3: PyTorch gradient backpropagation on the tensor cores!

Part 4: Value Functions, Q-Functions & Bellman Self-Consistency

(Slides 22–26 & Spinning Up ch07) In *Spinning Up in Deep RL*, Joshua Achiam emphasizes that value functions are central because they obey **Bellman Self-Consistency**: the value of your starting point equals the immediate reward plus the value of wherever you land.

4.1 The 4 Core Value Functions

Function	Mathematical Definition	Physical Role in Robotic Manipulation
On-Policy State Value $V^\pi(s)$	$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$	Expected total cut score if the knife is currently at state s and continues following current policy π .
On-Policy Action Value $Q^\pi(s, a)$	$\mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$	Expected score if the knife takes specific action a right now, and then continues following policy π .
Optimal State Value $V^*(s)$	$\max_{\pi} V^\pi(s) = \max_a Q^*(s, a)$	The absolute highest achievable cut quality possible from state s under the perfect policy π^* .
Optimal Action Value $Q^*(s, a)$	$\max_{\pi} Q^\pi(s, a)$	The highest achievable score starting from state s , taking action a , and acting optimally thereafter.

4.2 The Bellman Equations

Bellman Expectation Equation for V:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s), s' \sim \mathcal{P}(\cdot|s,a)} [r(s, a) + \gamma V^\pi(s')]$$

Bellman Optimality Equation for Q:

$$Q^*(s, a) = \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s,a)} \left[r(s, a) + \gamma \max_{a'} Q^*(s', a') \right]$$

4.3 Theoretical Proof: Bellman Contraction Mapping & Geometric Convergence

Why does repeatedly applying the Bellman equation converge to the true value function? Define the **Bellman Expectation Operator** $\mathcal{T}^\pi : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$:

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a | s) \left[\mathcal{R}(s, a) + \gamma \sum_{s'} \mathcal{P}(s' | s, a) V(s') \right]$$

THEOREM: Γ -CONTRACTION IN INFINITY NORM

For any two arbitrary value functions $V_1, V_2 \in \mathbb{R}^{|S|}$:

$$\|\mathcal{T}^\pi V_1 - \mathcal{T}^\pi V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$$

Proof: For any state s :

$$\begin{aligned} |(\mathcal{T}^\pi V_1)(s) - (\mathcal{T}^\pi V_2)(s)| &= \left| \gamma \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s, a) [V_1(s') - V_2(s')] \right| \\ &\leq \gamma \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s, a) |V_1(s') - V_2(s')| \leq \gamma \|V_1 - V_2\|_\infty \sum_a \pi(a|s) \sum_{s'} \mathcal{P}(s'|s, a) \end{aligned}$$

Since probabilities sum to 1, the sums evaluate to 1:

$$|(\mathcal{T}^\pi V_1)(s) - (\mathcal{T}^\pi V_2)(s)| \leq \gamma \|V_1 - V_2\|_\infty$$

Taking the supremum over all s gives $\|\mathcal{T}^\pi V_1 - \mathcal{T}^\pi V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty$.

By the **Banach Fixed-Point Theorem**, any contraction mapping on a complete metric space possesses a **unique fixed point** V^π such that $\mathcal{T}^\pi V^\pi = V^\pi$, and iterative application converges at an exponential geometric rate:

$$\|V_k - V^\pi\|_\infty \leq \gamma^k \|V_0 - V^\pi\|_\infty$$

4.4 Matrix Formulation & Closed-Form Analytical Solution

In finite/tabular MDPs with $|\mathcal{S}|$ states, the Bellman expectation equation can be written as a compact matrix-vector linear equation:

$$V^\pi = R^\pi + \gamma P^\pi V^\pi$$

where $R^\pi \in \mathbb{R}^{|\mathcal{S}|}$ has entries $R^\pi(s) = \sum_a \pi(a|s) \mathcal{R}(s, a)$, and $P^\pi \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{S}|}$ has entries $P^\pi(s, s') = \sum_a \pi(a|s) \mathcal{P}(s'|s, a)$. Rearranging gives:

$$(I - \gamma P^\pi) V^\pi = R^\pi \implies V^\pi = (I - \gamma P^\pi)^{-1} R^\pi$$

Because P^π is a row-stochastic matrix (eigenvalues ≤ 1) and $\gamma < 1$, the matrix $(I - \gamma P^\pi)$ is strictly invertible!

SPINNING UP BUG ALERT: THE [N] VS [N, 1] TENSOR BROADCASTING TRAP

Joshua Achiam highlights this as the #1 silent bug in RL implementations: If `rewards` has shape `[N]` (1D) and `next_values` has shape `[N, 1]` (2D), computing `targets = rewards + gamma * next_values` in PyTorch does NOT raise an error! Instead, Python broadcasts the two shapes into an **[N, N] matrix**! The Critic trains on random pairwise broadcasted values, loss stays finite, but the policy never learns. Always ensure shapes are strictly aligned using `.view(-1, 1)` or `.squeeze()`.

4.5 The Deadly Triad in Value Estimation

Richard Sutton and Andrew Barto formalized the **Deadly Triad**: when an RL algorithm combines three elements simultaneously, value estimation can diverge to infinity:

1. **Function Approximation:** Using neural networks $V_\phi(s)$ instead of tabular lookup tables.
2. **Bootstrapping:** Updating value estimates based on other value estimates: $y = r + \gamma V_\phi(s')$ (as in TD learning and Bellman equations).
3. **Off-Policy Training:** Training on data generated by an older or different behavior policy $\beta(a|s) \neq \pi(a|s)$ (as in replay buffers).

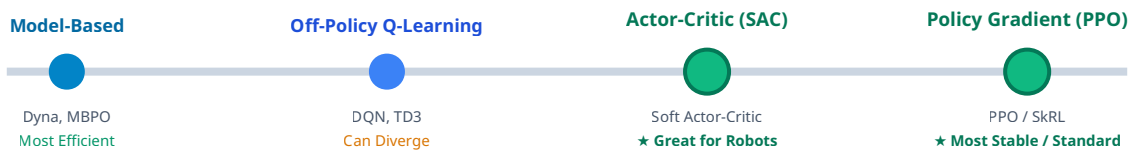
Understanding the Deadly Triad is why modern algorithms require specific stabilization anchors: target networks, double Q-learning, and clipping (explored in Lectures 8 and 10).

```
# Complete Safe Bellman Target Implementation
import torch

def compute_bellman_targets(rewards: torch.Tensor, next_values: torch.Tensor, dones: torch.Tensor, gamma: float):
    rewards = rewards.view(-1, 1)
    next_values = next_values.view(-1, 1)
    dones = dones.view(-1, 1)
    assert rewards.shape == next_values.shape == dones.shape, \
        f"Shape mismatch: {rewards.shape} vs {next_values.shape}"
    return rewards + gamma * (1.0 - dones) * next_values
```

Part 5: Taxonomy of RL Algorithms — Which One for Your Robotics Setup?

(Slides 27–41) Levine organizes all model-free algorithms along two fundamental axes: **Sample Efficiency** and **Stability**.



5.1 The Two Fundamental Trade-Offs

Dimension	Sample Efficiency	Stability & Convergence
On-Policy (PPO)	Lowest: Discards data after each batch. Needs millions of simulator steps.	Highest: True gradient ascent on clipped objective. Extremely forgiving to tune. (<i>Isaac Lab compensates via 4,096 parallel GPU envs!</i>)
Off-Policy (SAC)	High: Re-uses old trajectories from a replay buffer over and over.	Moderate: Needs careful tuning of entropy temperature α and target network updates to avoid overestimating Q-values.
Pure Q-Learning (DQN)	Moderate	Unstable on Continuous Arms: Computing argmax over continuous actions (torques) is mathematically intractable.
Model-Based RL	Highest: Tries to learn a neural network predicting physics transitions.	Brittle: Compounding model simulation errors lead to physical exploitation and catastrophic real-world failure.

Part 6: Interactive Tablet Self-Test Quiz

Question 1: If an environment state consists only of the knife's 3D Cartesian position $[x, y, z]$, does this satisfy the Markov property for cutting soft fruit?

Answer: No. A static position does not tell the robot whether the knife is accelerating downward or moving laterally, nor does it capture the accumulated elastic strain inside the tomato cuticle. To satisfy the Markov property, the state must include knife velocity ($\dot{z}$), lateral sawing speed, and TacBlade force derivatives ($\dot{F}_z$).

Question 2: In Bellman equations, what is the conceptual difference between $V^\pi(s)$ and $Q^\pi(s, a)$?

Answer: $V^\pi(s)$ is the expected return from state s when following policy π immediately. $Q^\pi(s, a)$ evaluates taking a specific designated action a first, and only following policy π on subsequent steps. The difference $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ tells us whether action a was superior to the policy's average choice.

Question 3: Why does Isaac Lab prefer on-policy PPO over off-policy SAC, despite PPO being less sample-efficient?

Answer: On GPU simulators like Isaac Lab, sample generation is virtually free (generating 260,000 steps in 50 ms across 4,096 parallel environments). PPO's stability, lack of memory-heavy replay buffers, and clipped objective allow straightforward scaling to massive parallel streams, whereas off-policy SAC causes GPU memory bottlenecks when managing replay buffers for 4,096 concurrent environments.

Part 7: Thesis Defense Master Cheatsheet (Lecture 4 Focus)**QUESTION: "WHY DID YOU USE REINFORCEMENT LEARNING INSTEAD OF STANDARD MODEL PREDICTIVE CONTROL (MPC)?"**

The Flawless Answer: "MPC requires an exact real-time analytical physics model of the tissue contact dynamics: $\dot{x} = f(x, u)$. When slicing soft deformable fruits or biological tissues, finite element modeling (FEM) or non-linear viscoelastic constitutive laws cannot be computed in real time at 100 Hz. RL bypasses the need for an explicit analytical model: it learns a parameterized feedback policy $\pi_\theta(a|s)$ entirely through simulated interaction and evaluative reward feedback, while training in Isaac Lab across thousands of parallel environments."

QUESTION: "WHY IS YOUR POLICY TRAINED ON VALUE FUNCTIONS RATHER THAN PURE MONTE CARLO ROLLOUTS?"

The Flawless Answer: "Pure Monte Carlo rollouts (like REINFORCE) exhibit massive gradient variance because they must sum rewards all the way to episode termination ($T = 500$). By utilizing Bellman Self-Consistency, the Critic $V_\phi(s)$ provides a 1-step bootstrapping target: $y_t = r_t + \gamma V_\phi(s_{t+1})$. This dramatically shrinks variance, enabling stable training with orders of magnitude fewer environment interactions."